

AI Expense Tracker

Architecture Document

v2.0 · May 2026

Agent-Centric · Tools + Rules · Local-First · PWA

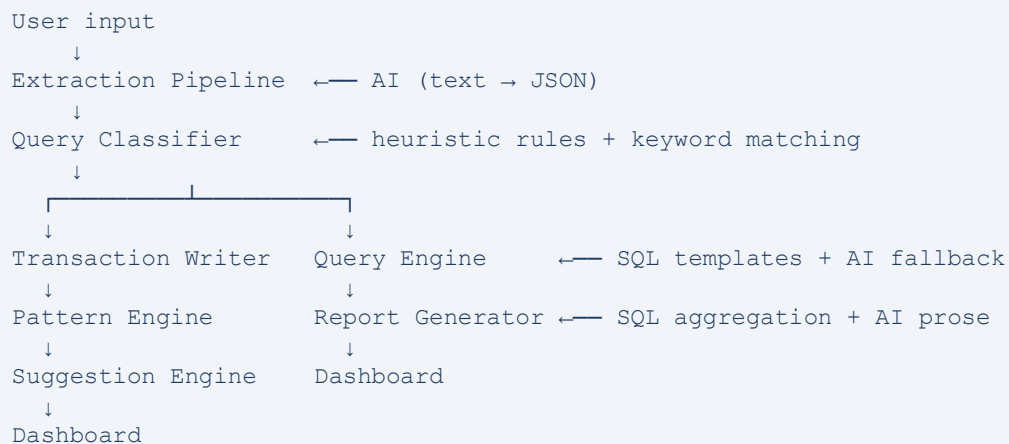
v2.0 change: Architecture rethought from extraction-pipeline model to AI-agent model. The AI is now the application logic. You write tools (thin DB functions) and rules (system prompt).
Result: ~45% less code, no query classifier, no pattern engine, no SQL template library.

The Paradigm Shift

v1.0 treated AI as a utility — one step in a deterministic pipeline. v2.0 inverts this. The AI is the pipeline. You write only the edges: database tools and behavioural rules.

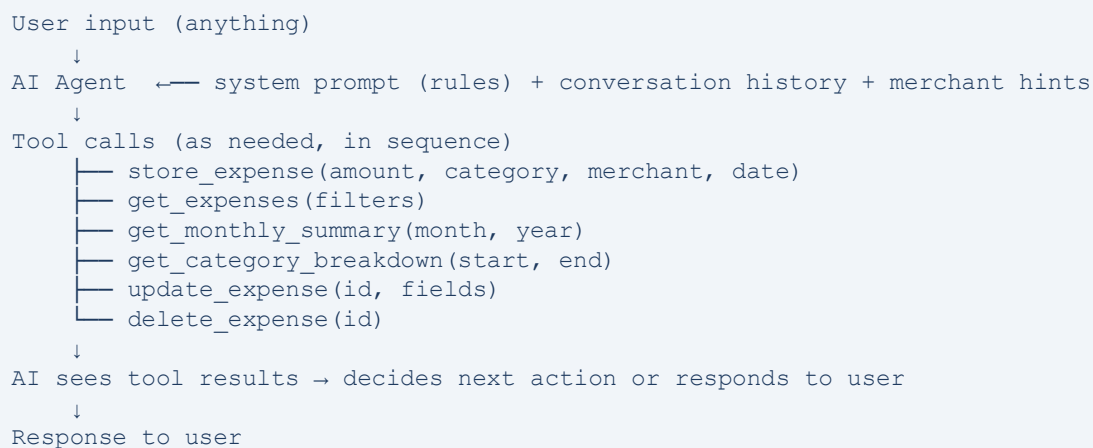
v1.0 — AI as NLP Utility

You wrote all the boxes. AI was just the text-parsing step inside the extraction pipeline.



v2.0 — AI as Application Logic

You write tools and rules. AI writes the logic at runtime.



What This Eliminates

Component	v1.0 Approach	v2.0 Approach	Status
Extraction pipeline	Prompt → JSON → Zod → parse	AI calls store_expense() directly	Gone
Query classifier	Heuristics + keyword matching	AI decides from context	Gone
SQL template library	Pre-built queries for common questions	AI constructs tool calls naturally	Gone
Report generator	SQL aggregation + separate AI prose call	AI calls summary tools and writes in one pass	Gone
user_rules table	Stored rules evaluated per transaction	Rules live in system prompt	Gone
Pattern engine	Scheduled SQL statistical heuristics	AI calls get_expenses and reasons over data	Moved
Suggestion engine	suggestions table + ranking + triggers	AI reasons + scheduler triggers the check	Moved
Merchant memory	merchant_category_map table (complex)	merchant_hints table (4 columns, simple)	Reduced
Schema complexity	8 tables	4 tables	Reduced
Dashboard / charts	React + Recharts + SQL	Unchanged — still React + Recharts + SQL	Kept

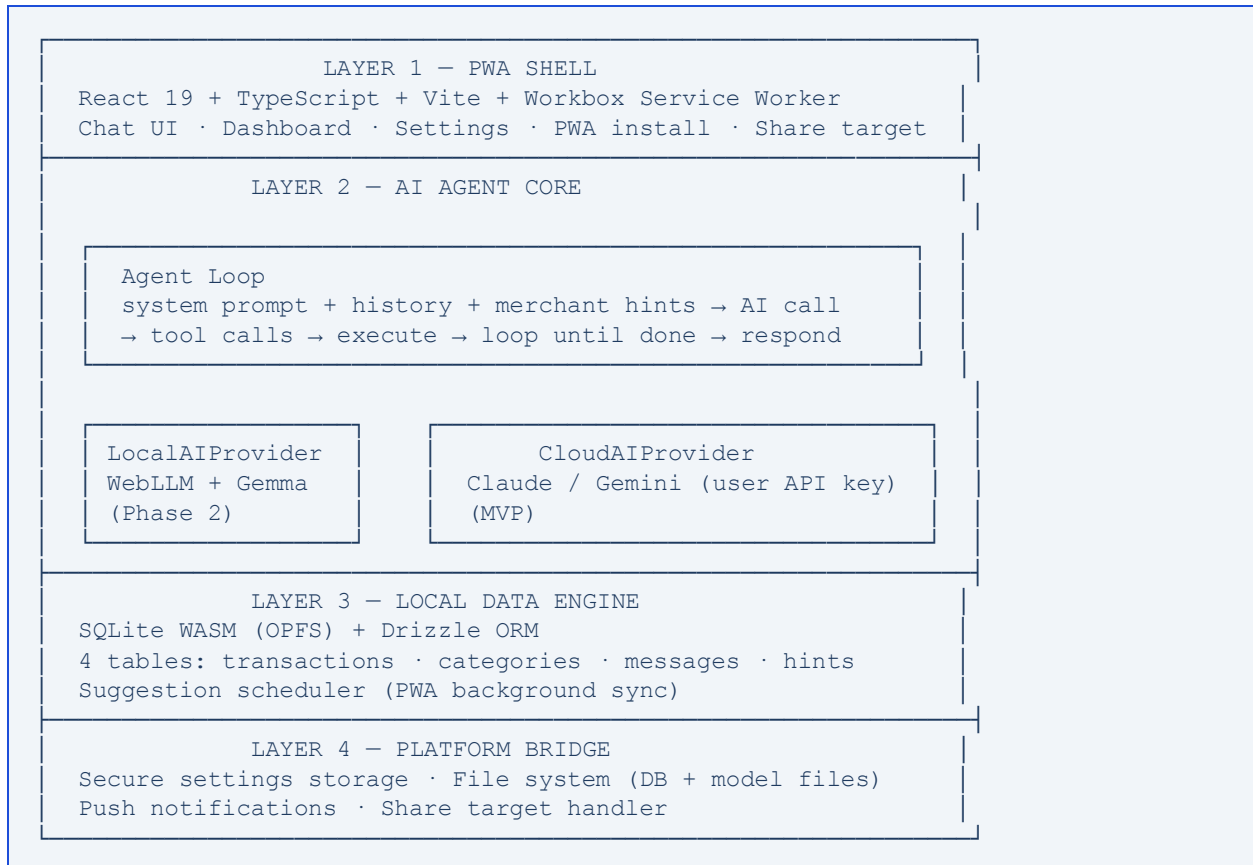
Architectural Decisions Log

All decisions from v1.0 that remain valid, plus new decisions introduced by the agent-centric shift.

Decision	Choice	Rationale
Platform	PWA (React + Vite)	No native toolchain. Instant install on Android. Widget deferred to Phase 4.
Data location	Local-first (SQLite OPFS)	Financial data is sensitive. Fully offline. User owns data. No cloud DB in MVP.
User scope	Single user, no auth	Building for self first. Auth adds weeks for zero MVP value.
Application logic	AI agent + tool calls	AI handles extraction, classification, querying, and reporting. You write tools and rules.
AI providers	Cloud (user key) → Local Phase 2	Cloud first — Claude/Gemini function calling is reliable. WebLLM Phase 2 after validation.
Backend	None	User-provided API key means no proxy server is needed. Calls go browser → AI API directly.
Merchant memory	merchant_hints table (4 cols)	Context window cannot hold 6 months of history. Lightweight table injected into each prompt.
Time-based suggestions	Suggestion scheduler (separate)	AI cannot self-trigger. PWA background sync wakes the agent on schedule.
Confirmation UX	Save-and-inform + undo	Faster than confirm-before-save. Natural correction via chat. One-tap undo in message bubble.
Dashboard	Hybrid: SQL charts + AI insights	Charts are SQL + Recharts. AI generates the insight paragraph at top of dashboard.
Context window	Last 20 messages	Good balance of accuracy vs token cost. Sufficient for multi-turn clarification.
Sync	Deferred Phase 2 (Turso)	Drizzle schema unchanged for libSQL migration. sync_metadata table stubbed only.

System Overview

A pure client application. No server. No cloud database. No authentication layer. All data lives on device. The only network calls are to the user's chosen AI provider API.



Layer 2 — AI Agent Core

Tool definitions · System prompt · Agent loop · Provider interface

Tool Definitions

Tools are the only interface between the AI and the database. Each tool is a thin, named wrapper over a Drizzle query. The AI decides which tools to call and in what order — you never write that logic.

Tool	Parameters	Purpose
store_expense	amount (rupees), category, merchant?, date?, note?	Insert a new expense transaction
store_income	amount (rupees), source, date?, note?	Insert a new income transaction
get_expenses	category?, start_date?, end_date?, merchant?, limit?	Query transactions with optional filters
get_monthly_summary	month (1–12), year	Aggregated totals and category breakdown for a month
get_category_breakdown	start_date, end_date	Per-category spend totals for any date range
get_recent_transactions	count? (default 10)	Most recent N transactions — used for context and corrections
update_expense	transaction_id, amount?, category?, merchant?, note?	Correct an existing transaction (partial update)
delete_expense	transaction_id	Soft-delete a transaction (is_deleted = 1)
get_budget_status	none	Current month spend vs budget per category

Every tool returns a typed result. The tool executor maps tool names to Drizzle functions. Adding a new capability means adding one tool definition and one Drizzle function — nothing else.

Tool Executor

The bridge between AI tool calls and the database. A pure dispatch function — no logic of its own.

```
// core/agent/tool-executor.ts
async function executeTool(
  call: ToolCall
): Promise<ToolResult> {
  switch (call.name) {
    case "store_expense":
      return db.insertTransaction({
        ...call.args,
        amount: rupeesToPaise(call.args.amount), // always convert at boundary
        type: "expense",
        id: nanoid(),
        created_at: Date.now(),
        updated_at: Date.now(),
        is_deleted: 0,
      });

    case "get_expenses":
      return db.queryTransactions(call.args);

    case "get_monthly_summary":
      return db.getMonthlySummary(call.args.month, call.args.year);

    case "get_category_breakdown":
      return db.getCategoryBreakdown(call.args.start_date, call.args.end_date);

    case "get_recent_transactions":
      return db.getRecent(call.args.count ?? 10);

    case "update_expense":
      return db.updateTransaction(call.args.transaction_id, {
        ...call.args,
        amount: call.args.amount
          ? rupeesToPaise(call.args.amount)
          : undefined,
        updated_at: Date.now(),
      });

    case "delete_expense":
      return db.softDelete(call.args.transaction_id);

    case "get_budget_status":
      return db.getBudgetStatus();

    default:
      return { error: `Unknown tool: ${call.name}` };
  }
}
```

Agent Loop

The entire application intelligence — about 50 lines. This replaces the extraction pipeline, query classifier, pattern engine, suggestion engine, query template library, and report generator from v1.0.

```
// core/agent/agent.ts
async function processMessage(
  userMessage: string,
  history: Message[],
  provider: AIProvider
): Promise<AgentResponse> {

  // Inject merchant hints for cross-session memory
  const hints = await db.getMerchantHints(30); // top 30 by frequency
  const context = buildSystemPrompt(hints); // system prompt + hint list

  const messages: ChatMessage[] = [
    { role: "system", content: context },
    ...history.slice(-20), // last 20 messages only
    { role: "user", content: userMessage },
  ];

  let response = await provider.chat(messages, TOOL_DEFINITIONS);

  // Agentic loop — AI may call multiple tools in sequence
  // e.g. "auto 25, chai 15, lunch 120" → 3 store_expense calls
  while (response.hasToolCalls) {
    const toolResults = await Promise.all(
      response.toolCalls.map(executeTool)
    );

    messages.push(
      { role: "assistant", content: response.raw },
      { role: "tool", content: toolResults }
    );

    response = await provider.chat(messages, TOOL_DEFINITIONS);
  }

  // Persist assistant message + any tool calls to messages table
  await db.saveMessage({
    id: nanoid(),
    role: "assistant",
    content: response.content,
    tool_calls: response.toolCalls ?? null,
    created_at: Date.now(),
  });

  return {
    text: response.content,
    toolsUsed: response.toolCalls ?? [],
  };
}
```


System Prompt — Behavioural Rules

Product behaviour lives here, not in code. Changing how the app behaves means editing this file — not touching the agent loop or tools.

You are a personal expense tracking assistant for a single user.
You help log, query, and understand their spending through
natural conversation. Be brief, casual, and efficient.

— LOGGING RULES —

When the user mentions spending money:

1. Call `store_expense()` immediately — do not ask first
2. Confirm after: "Saved: Food ₹120 at Chai Point · tap to undo"
3. If multiple items in one message, call `store_expense` for each
4. If amount is missing or ambiguous — ASK, never guess
5. "5k" = ₹5,000 | "1.5L" = ₹1,50,000 | bare "5" = ask

Category rules:

- Pick the single best-fit category silently
- Available: Food, Transport, Shopping, Bills & Utilities, Rent, Health, Education, Entertainment, Travel, Groceries, Personal Care, Gifts, Subscriptions, Other
- If genuinely ambiguous, ask — don't default to Other
- Corrections: if user says "that was Transport not Food", call `update_expense` AND note the correction for future entries

— QUERYING RULES —

When the user asks about spending:

1. Use `get_expenses` or `get_monthly_summary` — don't guess from memory
2. Present numbers cleanly: "You spent ₹8,420 on food in April"
3. Comparisons: fetch both periods, calculate the delta
4. "How am I doing this month?" → call `get_budget_status`

— PATTERN AWARENESS —

- When called by the scheduler (message starts with `[SCHEDULER]`): call `get_recent_transactions(30)` and check for repeated patterns
- If same category + similar amount appears 3+ times same time/day: suggest: "Looks like your usual auto fare — want me to log ₹25?"
- Do not suggest patterns the user has previously dismissed

— REPORTS —

When asked for a monthly report:

1. Call `get_monthly_summary(month, year)`
2. Call `get_category_breakdown(start, end)`
3. Write: top category, biggest single expense, savings rate, one notable observation vs prior month

— TONE & CONSTRAINTS —

- No financial advice — observations only

- Currency always INR (₹). Never show paise to user.
- Amounts stored in paise internally — never expose this
- Maximum one clarifying question per message
- Never fabricate numbers — always use tools for data

Merchant Hints — Cross-Session Memory

The context window holds only 20 messages — roughly one session. Merchant→category mappings built over months cannot live in conversation history. The `merchant_hints` table is the fix: a small lookup injected into every system prompt.

```
-- merchant_hints: populated automatically by tool executor
-- whenever store_expense or update_expense is called
CREATE TABLE merchant_hints (
  canonical_name TEXT PRIMARY KEY,    -- lowercased, trimmed merchant name
  category       TEXT NOT NULL,       -- last confirmed category
  use_count      INTEGER DEFAULT 1,   -- frequency for ranking
  last_used_at   INTEGER NOT NULL
);

-- Top 30 hints are fetched before every agent call and
-- appended to the system prompt as:
-- "Known merchants: zepto→Groceries, swiggy→Food, ola→Transport ..."
```

Hint injection is invisible to the user. The AI receives the hint list as part of its context and applies it silently.

When the user corrects a categorisation, `update_expense` also updates `merchant_hints` — the correction is permanent from the next message onward.

Suggestion Scheduler

The AI cannot self-trigger. Time-based suggestions require a mechanism to wake the agent on schedule. In a PWA, this is the Background Sync API combined with periodic local checks.

```
// PWA service worker — periodic background sync
self.addEventListener('periodicsync', async (event) => {
  if (event.tag === 'suggestion-check') {
    event.waitUntil(runSuggestionCheck());
  }
});

async function runSuggestionCheck() {
  const hour = new Date().getHours();
  const dow  = new Date().getDay();    // 0=Sun...6=Sat
  const message = `[SCHEDULER] hour=${hour} dow=${dow}. Check for patterns.`;
```

```
// Runs the full agent loop with a synthetic scheduler message
// AI calls get_recent_transactions, spots patterns, sends notification
const result = await agentLoop(message, [], activeProvider);

if (result.hasSuggestion) {
  self.registration.showNotification("Expense Tracker", {
    body: result.suggestionText,      // "Your usual auto fare - ₹25?"
    data: { type: 'suggestion', prefill: result.suggestionData },
    actions: [{ action: 'log', title: 'Log it' }, { action: 'dismiss', title:
'Dismiss' }]
  });
}
}
```

Layer 3 — Local Data Engine

SQLite WASM (OPFS) + Drizzle ORM · 4 tables

SQLite WASM Setup

- Library: @sqlite.org/sqlite-wasm — official SQLite WASM build
- Persistence: OPFS (Origin Private File System) — real file, not IndexedDB
- Threading: Web Worker — never blocks the main thread
- ORM: drizzle-orm/sqlite-core — identical API to Node better-sqlite3
- Migrations: Drizzle Kit, run in Web Worker on app boot

Database Schema — 4 Tables

Reduced from 8 tables in v1.0. *patterns, suggestions, user_rules, and merchant_category_map* are all gone.

transactions

```
transactions {
  id          TEXT      PRIMARY KEY      -- nanoid()
  amount      INTEGER NOT NULL           -- paise (₹1 = 100). NEVER floats.
  type        TEXT      NOT NULL         -- 'expense' | 'income'
  category    TEXT      NOT NULL         -- direct string, FK to
categories.name
  merchant    TEXT                               -- nullable
  note        TEXT                               -- nullable
  date        TEXT      NOT NULL         -- 'YYYY-MM-DD'
  created_at  INTEGER NOT NULL           -- Unix ms
  updated_at  INTEGER NOT NULL
  is_deleted  INTEGER NOT NULL DEFAULT 0 -- soft delete always
}
```

amount is always stored in paise. The tool executor converts at the boundary (rupeesToPaise on write, paisoToRupees on read). No float ever enters the DB.

categories

```
categories {
  id          TEXT      PRIMARY KEY
  name        TEXT      NOT NULL UNIQUE
  icon        TEXT                               -- emoji
  budget_amount INTEGER           -- paise, nullable — monthly limit
  is_default  INTEGER NOT NULL DEFAULT 1
```

```

    sort_order    INTEGER
  }

```

messages

```

messages {
  id          TEXT    PRIMARY KEY
  role        TEXT    NOT NULL    -- 'user' | 'assistant'
  content     TEXT    NOT NULL    -- display text
  tool_calls  TEXT          -- JSON array of tool calls, nullable
  created_at  INTEGER NOT NULL

  -- Only last 20 rows are sent to AI per call (ORDER BY created_at DESC LIMIT 20)
  -- Older rows are retained for UI chat history but excluded from AI context
}

```

merchant_hints

```

merchant_hints {
  canonical_name TEXT    PRIMARY KEY    -- lowercased, trimmed
  category       TEXT    NOT NULL       -- last confirmed category
  use_count      INTEGER NOT NULL DEFAULT 1
  last_used_at   INTEGER NOT NULL

  -- Top 30 by use_count injected into system prompt before every AI call
  -- Updated by tool executor on every store_expense / update_expense call
}

```

sync_metadata (stub — Phase 2 only)

```

sync_metadata {
  id          TEXT PRIMARY KEY DEFAULT 'singleton'
  device_id   TEXT NOT NULL          -- random UUID at install
  last_synced_at INTEGER             -- null until sync configured
  turso_db_url TEXT                  -- null until sync configured
}

```

Data Flows

Six core journeys through the agent

Flow 1 — Simple Expense Entry

User: "bought cooler for 5k"

1. ChatInput → processMessage("bought cooler for 5k", history, provider)
2. System prompt + last 20 messages + merchant hints assembled
3. AI call → response has tool call:
store_expense({ amount: 5000, category: "Shopping",
merchant: null, date: "2026-05-17" })
4. tool executor: rupeesToPaise(5000) = 500000 → Drizzle insert
5. merchant_hints: no merchant → no hint update
6. AI sees tool result → no more tool calls needed
7. AI responds: "Saved: Shopping ₹5,000 – cooler · tap to undo"
8. Message saved to messages table
9. TanStack Query invalidates ['transactions', 'dashboard']
10. Dashboard re-renders

Flow 2 — Multi-Item Entry

User: "auto 25, chai 15, lunch 120"

1. processMessage called with full message
2. AI identifies 3 items → makes 3 sequential tool calls:
store_expense({ amount: 25, category: "Transport", merchant: null })
store_expense({ amount: 15, category: "Food", merchant: null })
store_expense({ amount: 120, category: "Food", merchant: null })
3. Each executes, each result returned to AI
4. AI responds: "Saved 3 entries: Transport ₹25, Food ₹15, Food ₹120"

Note: parallel Promise.all() used for independent tool calls
sequential calls used only when later calls depend on earlier results

Flow 3 — Correction via Chat

User: "that cooler should be Home Appliances not Shopping"

1. AI calls get_recent_transactions(5) to find the entry
2. Identifies transaction ID for the ₹5,000 cooler entry

3. Calls `update_expense({ transaction_id: "xyz", category: "Home Appliances" })`
4. Tool executor: Drizzle update + merchant_hints update if merchant present
5. AI responds: "Updated – cooler now in Home Appliances ✓"

No separate "edit flow" needed. Corrections are just chat.

Flow 4 — Conversational Query

User: "how much did I spend on food last month?"

1. AI identifies this as a query – no `store_expense` call
2. Calls `get_monthly_summary(4, 2026)` -- April
3. Receives full summary object
4. Calls `get_category_breakdown("2026-04-01", "2026-04-30")`
5. Responds: "You spent ₹8,420 on food in April – 34% of your total spend"

Comparison query:

User: "how does that compare to March?"

→ AI calls `get_monthly_summary(3, 2026)`, calculates delta, responds
"March was ₹6,200 – April was 36% higher"

Flow 5 — Scheduler-Triggered Suggestion

[Background Sync fires at 7:30 AM Wednesday]

1. Service worker sends synthetic message to agent loop:
"[SCHEDULER] hour=7 dow=3. Check for patterns."
2. AI calls `get_recent_transactions(30)`
3. Sees: auto ₹25 logged on 6 of last 8 weekday mornings
4. Returns suggestion flag + text
5. Service worker shows notification:
"Your usual auto fare – ₹25? [Log it] [Dismiss]"
6. User taps "Log it" → `store_expense` called directly,
no AI call needed for the confirmation tap

Flow 6 — Monthly Report

User: "show me my April report"

1. AI calls `get_monthly_summary(4, 2026)`
2. AI calls `get_category_breakdown("2026-04-01", "2026-04-30")`
3. AI also calls `get_monthly_summary(3, 2026)` for comparison

4. Writes report in one response:

"April summary

Income ₹80,000 · Spent ₹34,200 · Saved ₹45,800 (57%)

Top categories: Food ₹8,420 · Rent ₹12,500 · Transport ₹3,100

Notable: Food spend was 36% higher than March — likely the
Goa trip expenses. Savings rate is your best in 4 months."

No separate report generation pipeline. No separate AI call for prose.

Project Structure

What you actually build

Source Tree

```

src/
├── app/
│   ├── App.tsx                # Router root
│   └── routes/
│       ├── index.tsx          # Chat view (primary)
│       ├── dashboard.tsx      # Dashboard view
│       └── settings.tsx       # API key + provider settings
├── core/
│   ├── agent/
│   │   ├── agent.ts           # processMessage - the whole app logic
│   │   ├── tools.ts           # TOOL_DEFINITIONS array
│   │   ├── tool-executor.ts   # Tool name → Drizzle function dispatch
│   │   └── system-prompt.ts   # SYSTEM_PROMPT + buildSystemPrompt(hints)
│   ├── providers/
│   │   ├── types.ts           # AIProvider interface
│   │   ├── cloud.ts           # CloudAIProvider (Claude + Gemini)
│   │   └── local.ts           # LocalAIProvider (WebLLM - Phase 2)
│   ├── db/
│   │   ├── schema.ts          # Drizzle schema - 4 tables
│   │   ├── worker.ts          # SQLite WASM Web Worker setup
│   │   ├── client.ts          # Typed DB client (thin Drizzle wrappers)
│   │   └── migrations/        # Drizzle Kit output
│   └── domain/
│       ├── money.ts           # Money type, rupeesToPaise, paiseToRupees
│       └── types.ts           # Transaction, Category, Message, Hint
├── features/
│   ├── chat/
│   │   ├── ChatView.tsx       # Message list + input box
│   │   ├── MessageBubble.tsx  # Single message with undo action
│   │   └── chat.store.ts       # Zustand - input state, sending flag
│   ├── dashboard/
│   │   ├── DashboardView.tsx  # Header insight + charts
│   │   ├── CategoryChart.tsx  # Recharts donut
│   │   └── MonthSummary.tsx    # Income / spent / saved numbers
│   └── settings/
│       └── SettingsView.tsx    # Provider picker + API key input
└── shared/
    ├── components/            # Button, Input, Modal, etc.
    └── hooks/                 # useTransactions, useMonthlySummary

```

```
|  
├─ sw/  
│   └─ service-worker.ts           # Workbox config + suggestion scheduler  
├─ styles/  
│   └─ globals.css
```

What Is Notably Absent

- No `extractionService.ts` — the agent loop handles extraction
- No `queryClassifier.ts` — AI decides intent from context
- No `patternEngine.ts` — AI reasons over `get_recent_transactions` results
- No `suggestionStore.ts` — suggestions table does not exist
- No `reportGenerator.ts` — AI calls summary tools and writes prose directly
- No `userRulesEngine.ts` — rules live in the system prompt

Complete Tech Stack

Every library and why

Frontend

Package	Version	Purpose
react	19.x	UI framework. Client-only — no SSR.
typescript	5.x	Strict mode. Domain types (Money, Transaction) defined once, used everywhere.
vite	6.x	Build tool. Fast HMR. vite-plugin-pwa for service worker generation.
vite-plugin-pwa	latest	Manifest + Workbox service worker. Handles install prompt.
@tanstack/react-query	5.x	Async state for local DB reads. Invalidation triggers re-render after tool calls.
@tanstack/react-router	1.x	Type-safe file-based routing.
zustand	5.x	UI-only ephemeral state — chat input buffer, modal state.
tailwindcss	4.x	Layout only. shadcn handles component styling.
shadcn/ui	latest	Radix-based accessible components. Copy-paste, not a runtime dependency.
recharts	2.x	Dashboard charts — donut for categories, bar for monthly trend.
date-fns	3.x	Date arithmetic — month ranges, relative date display.
nanoid	5.x	ID generation for transactions and messages.

Data Layer

Package	Version	Purpose
@sqlite.org/sqlite-wasm	latest	Official SQLite WASM build. OPFS persistence. Web Worker threading.
drizzle-orm	0.40+	Type-safe ORM. SQLite dialect. Same API as Node better-sqlite3.

Package	Version	Purpose
drizzle-kit	latest	Migration generation. Schema changes managed through standard migration files.
zod	3.x	Runtime validation of tool call arguments before DB write.

AI Layer

Package	Version	Purpose
@anthropic-ai/sdk	latest	Anthropic SDK — browser-compatible. Direct API calls with user's own key.
@google/generative-ai	latest	Gemini SDK — browser-compatible. Direct API calls with user's own key.
@mlc-ai/web-llm	latest	WebGPU local LLM runtime. Gemma model. Phase 2 only.

Phase 2+ Libraries

Package	Version	Purpose
@libsql/client	latest	Turso/libSQL sync. Drizzle schema works unchanged with libSQL.
jspdf + autoTable	latest	Client-side PDF export for monthly reports.

Phase-wise Build Plan

Implementation order and effort estimates

MVP Build Order

Ordered by dependency. Each step is a usable checkpoint — the app gains capability incrementally.

#	Deliverable	Effort	Notes
1	Vite + React + TypeScript scaffold	1–2 hrs	Routing, Tailwind, shadcn base setup
2	SQLite WASM + OPFS + Web Worker	2–3 hrs	Validate OPFS works on target Android device early
3	Drizzle schema (4 tables) + seed categories	1 hr	Run migration, verify DB initialises
4	Domain types — Money, Transaction, Message	1 hr	rupeesToPaise, paiseToRupees, all domain types
5	Tool definitions + tool executor	2 hrs	All 9 tools with Drizzle implementations
6	AIProvider interface + CloudAIProvider	2 hrs	Claude first. Zod validation of tool call args.
7	Agent loop (processMessage)	2 hrs	The core. Multi-tool loop, message persistence.
8	Chat UI + message rendering	3–4 hrs	Message list, input, loading state, undo action
9	Dashboard — SQL charts + AI insight text	3–4 hrs	TanStack Query reads, Recharts, insight AI call
10	Settings — provider picker + API key	1–2 hrs	Encrypted key storage, provider switch
11	PWA — manifest + service worker + install	1–2 hrs	Workbox caching, install prompt, offline fallback

Estimated MVP total: 19–24 hours of focused work. Steps 1–7 (foundation + agent core) are the critical path — once step 7 works, the app is functional even without polished UI.

Phase 2 — Intelligence + Offline

- WebLLM integration — LocalAIProvider behind same AIProvider interface
- Gemma model download flow with progress screen on first use
- Turso sync — libSQL remote replica, sync_metadata table activated
- Suggestion scheduler — Background Sync API + notification actions
- Budget alerts — AI mentions budget status passively during entry

Phase 3 — Automation

- SMS parsing — Android notification listener via Web Share Target
- Draft queue — parsed SMSes shown for one-tap confirmation
- Voice input — Web Speech API → same processMessage pipeline
- Recurring auto-log — scheduler creates store_expense calls automatically

Phase 4 — Portability

- CSV + PDF export — client-side, no server
- Goal tracking — new goals table + AI coaching via system prompt addition
- Android home screen widget — Capacitor wrapper at this point

Design Principles

AI writes logic, you write boundaries Tools define what's possible. System prompt defines how to behave. Agent loop wires them together. Nothing else.	Local-first contract Every read and write hits local SQLite. Network is enhancement, not dependency. App boots and works fully offline.	Paise not rupees All monetary values stored as integers in paise. Display layer converts. Zero floating-point bugs in aggregations.
Soft deletes always No transaction is ever hard-deleted. <code>is_deleted</code> flag only. AI mistakes must be undoable in one tap.	One boundary conversion <code>rupeesToPaise</code> on write. <code>paiseToRupees</code> on read. The conversion happens exactly twice — in tool executor. Never elsewhere.	System prompt is product Adding a feature means adding a tool and a rule. Not a new pipeline, service, or table. The system prompt IS the product spec.

Open Questions

Resolved from v1.0 discussion. Listed here for reference.

Question	Decision	Rationale
Context window size	Last 20 messages	Good accuracy/cost balance. Sufficient for multi-turn clarification within a session.
Confirmation before saving	Save-and-inform + undo	Faster than confirm flow. Corrections are natural chat. One-tap undo in message bubble.
Dashboard approach	Hybrid (SQL + AI insight)	Charts are SQL + Recharts. AI writes insight paragraph. Fully AI dashboard would be slow and inconsistent.
WebLLM scope	Phase 2, not MVP	Cloud function calling is reliable. Local model function calling needs validation first. Design for cloud, test local in Phase 2.
Multi-currency	INR only for MVP	Store in paise. Add exchange_rate column in Phase 3 if needed for travel expenses.
Category editability	Add only, no delete	Deleting a category breaks historical data. Users can add custom categories. Default 14 cannot be deleted.

AI Expense Tracker — Architecture Document v2.0

Agent-Centric · Tools + Rules · 4 Tables · No Server · May 2026